

COMMERZA: Engineering a Production-Grade E-Commerce Architecture from Scratch

A Technical Case Study on Solo Architecture Generation, Enterprise Security Layering, and Advanced Human-AI Collaboration Mechanics

PRINCIPAL ENGINEER	SYSTEM STACK	DEVELOPMENT DURATION	PROJECT CONTEXT
Syed Ahmer Shah	Native PHP & MySQL	3 Months (Jan–Apr 2026)	GitHub Finish-Up-A-Thon

1. Project Background & Intent

In January 2026, the **Commerza** initiative was launched to build a resilient, production-grade e-commerce platform completely from the ground up without using monolithic modern frameworks (e.g., Laravel). The primary architectural goal was to uncover, master, and manually construct the core infrastructure patterns that frameworks typically obfuscate: transactional integrity, edge-case routing, high-density authentication models, security pipelines, and custom asset delivery loops.

Developed as a solo student project over three months by a 19-year-old engineer in Hyderabad, Pakistan, the project scaled rapidly from simple frontend components into a comprehensive environment composed of **80 tightly integrated core PHP files**, spanning 339 total tracked modules. The project culminated in an official submission to the *GitHub Finish-Up-A-Thon Challenge* and was successfully archived on May 5, 2026, following the thorough implementation of enterprise-level operational boundaries.

339	80	136	3 Months
TOTAL FILES TRACKED	CORE PHP SCRIPTS	VERIFIED COMMITS	TIMELINE BASELINE

2. Architectural Layout & Stack Topology

The platform relies on a strict baseline: ****PHP Native (MySQLi Prepared-Statement Layer)**** coupled with an ****Apache**** runtime container. To facilitate future technical transitions, a modular ****PDO Helper Interface**** was integrated into the initialization lifecycle, supporting smooth runtime database access migration.

Core Paradigm: Layout Isolation

The system splits application state into public client interfaces and privileged admin services, ensuring explicit network separation and strict path boundaries across execution contexts.

- **Storefront Entrypoints (Root Profile):** 24 functional public routing components handling layout renders, instant user states, cart logic, invoices, tracking, and localized SEO setups.
- **admin/ Module Context:** Houses the administrative command system, isolating secure UI views from underlying administrative endpoints (`admin/auth/` and `admin/api/`).
- **backend/ Core Architecture:** Contains critical application configurations, database connections (`backend/core/data.php`), background cron processes, performance reporting scripts, and security middlewares.
- **Frontend Assets:** Modular UI layers built with clean HTML5, custom utility-first responsive structures using Tailwind CSS/Bootstrap, and decoupled asynchronous interactive elements driven by jQuery.

3. Deep-Dive Security Posture

Commerza addresses the common security vulnerabilities that often plague vanilla PHP web applications by implementing a comprehensive, multi-layered threat mitigation matrix.

3.1 SQL Injection (SQLi) Structural Defense

Every single user-facing query and mutation route utilizes strict parameter serialization using native MySQLi prepared statements. String concatenation is barred from all dynamic request pipelines. To protect the database layer further, incoming identifiers are strictly cast to structural integer types, such as `id = (int)$_POST['id']`, and all action variables are filtered against rigid, server-defined white-lists prior to compilation.

3.2 Advanced Multi-Layer Hybrid CAPTCHA System

The user-interaction routes (contact requests, authentications, mutation loops) are guarded by a hybrid anti-automation mechanism implemented in `backend/security/security_helpers.php` :

1. **Automated Honeypot:** Intercepts naive automation scripts using a hidden field (`commerza_contact_website`) that must remain empty; any injection triggers an immediate drop sequence.
2. **Google reCAPTCHA v3 & v2 Dual-Engine:** Primary validations utilize reCAPTCHA v3 with a strict minimal validation floor threshold of $S_{\{min\}} = 0.65$ (with a hard engine configuration floor at 0.55). The backend aggressively validates token metadata, context strings, freshness, and hostname parity. The architecture automatically down-grades to a visible reCAPTCHA v2 checkbox if specific validation conditions or keys require it.
3. **Server-Side Cryptographic Fallback Challenge:** If network timeouts occur or token telemetry is low, a secure mathematical/logic challenge engine engages. It verifies user answers against a short-lived cryptographically signed token payload (nonce-bounded hash matching).

3.3 Cryptographic Integrity & Identity Lifecycles

Password security is handled entirely by `backend/security/security_helpers.php`. The architecture prioritizes native **Argon2id** (`PASSWORD_ARGON2ID`) algorithms, automatically falling back to **Bcrypt (Cost Factor 12)** if the

host environment lacks memory-hard optimization libraries. It continuously monitors security thresholds, dynamically trigger-hashing older tokens via runtime re-evaluation mechanisms.

Authentication states apply automated constraints, including a **90-Day Structural Lockout** on profile username modifications (`users.username_changed_at`), preventing account takeover spoofing. All verification workflows and temporary operational tokens use a strict 15-minute expiration window before automatic invalidation.

4. Critical Transactional & Checkout Infrastructure

The checkout and ordering subsystem contains advanced concurrency controls to prevent edge-case race conditions during multi-user simultaneous database changes.

```
// High-Density Checkout Concurrency Control - backend/order/checkout_processor.php
$db->begin_transaction(MYSQLI_TRANS_START_READ_WRITE);
try {
    // 1. Structural Row Locking to Block Concurrent Stock Deductions
    $stmt = $db->prepare("SELECT stock_quantity, status FROM products WHERE id = ? FOR UPDATE");
    $stmt->bind_param("i", $productId);
    $stmt->execute();
    $product = $stmt->get_result()->fetch_assoc();

    if ($product['stock_quantity'] < $requestedQuantity || $product['status'] !== 'active') {
        throw new Exception("Inventory exhaustion or listing deactivation during transaction window.");
    }

    // 2. Consume Idempotency Key to Prevent Duplicate Form Submissions
    $idempotencyCheck = $db->prepare("UPDATE transactions SET status = 'processing' WHERE key_hash = ? AND status = 'open'");
    // ... complete execution blocks ...
    $db->commit();
} catch (Exception $e) {
    $db->rollback();
    log_security_event("Checkout Failure", $e->getMessage());
}
```

The system also includes a configurable **High-Value Cash on Delivery (COD) Threshold Policy** (`COMMERZA_COD_OTP_THRESHOLD` , default 15,000 PKR). When a transaction exceeds this financial baseline, the engine locks the checkout pipeline and requires a 6-digit email OTP from the user before executing the order creation transaction.

5. Human-AI Collaborative Engineering Mechanics

A key factor in accelerating the project timeline from an estimated 8 months down to 3 months was the targeted integration of **GitHub Copilot**, utilizing a multi-model approach tailored to specific engineering requirements.

5.1 Multi-Model Selection Strategy

- **Claude 3.5 Sonnet:** The primary architecture asset. Used for structured code context aggregation, multi-file framework alignment, and rapid UI development across Bootstrap/Tailwind modules.
- **Claude 3.0 Opus:** Selected for complex algorithmic problems, including designing the 3-level security severity architecture, implementing multi-user transaction boundaries, and handling row locking.
- **GPT-4o/Codex Variants:** Deployed as a distinct verification layer, analyzing code blocks for logical edge cases and validation flaws.

5.2 Technical Architecture Evolution

The utilization of targeted AI support introduced patterns that went far beyond basic code generation. During the development of the automated email layer (`backend/mailer/mailer.php`), a simple failover suggestion evolved into a dual-route architecture. This system features built-in **duplicate suppression checks** to prevent split-brain delivery anomalies across separate primary and secondary SMTP gateways.

Dimension	Manual Baseline Baseline	Engineered Post-Copilot State
File Metrics	~8 basic isolated script layouts with manual links.	80 tracked PHP modules forming an interconnected platform.
Security Model	Basic string filtering and manual input escaping.	3-Tier Rigorous Framework: Baseline Filters → Sensitive Inputs → Critical Money Path Locking.
Notification Core	Single unauthenticated PHP <code>mail()</code> instantiation.	Resilient Transport Loop: Multi-route SMTP failover + 6 automated data engines.
Advanced Concepts	Basic acquaintance with standard PDO/MySQLi methods.	Enterprise Architecture Patches: Row-level stock locking, CSRF nonces, Argon2id, and token-based idempotency.

6. Administrative Operations & Platform Automation

The administration architecture includes extensive management capabilities designed to handle daily business operations safely and securely.

- **Granular Role Gating & Sub-Admin Lifecycles:** Super-admins can spin up sub-admin operational identities with custom access permissions. The system enforces **Immediate Session Revocation** across the network if a sub-admin account is suspended, deleted, or has critical profile metrics modified.
- **Product Trash & Data Recovery Loops:** Built-in soft-delete architectures allow admins to manage product life-cycles safely. Archived items are automatically filtered from the public storefront while remaining completely restorable from the admin console.
- **Automated Data Reporting Closures:** Built-in background scripts (including `send_engagement_reminders.php` and `monthly_profit_report.php`) generate structured summaries, compiling core metrics into automated analytics notifications for the engineering and business teams.

7. Automated Testing & CI Security Gates

To preserve system integrity across code revisions, a Continuous Integration suite was deployed using GitHub Actions via `.github/workflows/security-gate.yml`. This workflow evaluates system files against a strict syntax baseline and runs targeted static analysis checks on every push. When the `SECURITY_BASE_URL` environment variable is fully configured, the security pipeline executes dynamic probes across runtime endpoints to catch potential structural regressions before they reach deployment.

8. Conclusions & Strategic Key Architectural Takeaways

The Commerza project demonstrates that building a robust, production-ready system without heavy frameworks is fully achievable through disciplined, manual infrastructure design. By treating advanced development tools as architectural partners rather than simple code generators, a solo engineer can successfully design, test, and ship highly secure enterprise software patterns within an accelerated delivery window.